Source: docs\PAYMENTS.md | Export: 11 November 2026

Razorpay payments - Parinaya Sarees

This integration uses the existing orders, checkout review, product inventory, shipping, coupon and loyalty services. It does not create a second order system. COD continues working without Razorpay configuration. Real gateway TEST acceptance and LIVE activation are separate release gates; mocked tests do not establish either.

For the current endpoint audit, exact Vercel variables, worker setup and first TEST transaction checklist, use Razorpay TEST setup. The dated implementation record below remains historical evidence.

Credentials and activation

Only these three Razorpay credentials are needed from the client, delivered through the secure deployment environment, never chat, source files or frontend variables:

Server variable	Source
`RAZORPAY_KEY_ID`	Razorpay Dashboard → appropriate TEST/LIVE mode → API Keys
`RAZORPAY_KEY_SECRET`	The corresponding generated API key secret
`RAZORPAY_WEBHOOK_SECRET`	A separate strong secret selected when configuring that mode's webhook

Also configure `RAZORPAY_MODE=test` or `live`. Prefix validation rejects mixed-mode key IDs; there is no fallback from LIVE to TEST. The key secret and webhook secret are never returned to a browser. The public key ID is returned only to an eligible, authenticated checkout. Do not add any `VITE_RAZORPAY_*` variables.

For local TEST development, put server values in the ignored `.env.server.local` and run `npm run dev:api` with `npm run dev`. Set `RAZORPAY_TEST_ENVIRONMENT=true` for an explicitly configured non-production local environment, or `RAZORPAY_TEST_UIDS` to a comma-separated list of approved test UIDs. Do not use live accounts/orders as automated fixtures. TEST checkout is labelled.

For Production TEST acceptance, set TEST credentials and **only** approved UIDs in `RAZORPAY_TEST_UIDS`. Ordinary production accounts remain excluded. The local test environment switch cannot enable public TEST checkout in Production. Preview origins are deliberately not allowed; use loopback or approved production domains. In addition, `VERCEL_ENV=preview` explicitly rejects all payment endpoints and service operations before authentication/rate-limit writes or provider calls, even if credentials, a production Origin or the LIVE activation flag are copied into that deployment. COD does not use this guard. Keep payment secrets scoped to Production; the guard is additional protection, not a reason to broaden scopes.

LIVE remains disabled until the client has completed Razorpay account activation, provided LIVE keys, and explicitly confirmed activation. Then set `RAZORPAY_MODE=live` and `RAZORPAY_LIVE_ACTIVATED=true`. Remove test UID/environment flags. Do not set the activation flag simply because an API request succeeds.

Recovery worker is required before hosted payment initiation

Set `PAYMENT_WORKER_SECRET` to a random server-only value of at least 32 characters. Configure a reliable external scheduler to POST every minute to:

`https://www.parinayasarees.com/api/payments/worker`

Use `Authorization: Bearer <PAYMENT_WORKER_SECRET>`, `Content-Type: application/json` and a JSON body `{}`. For example, use a securely configured Cloud Scheduler HTTP job. Do not put its secret in command history, job names, source or logs. Scheduler provisioning is an external operations step, not an application deployment side effect.

The worker processes bounded due batches with a time budget, retries accepted events, reconciles ambiguous orders and expires eligible reservations. Its server-only `paymentOperations/worker` heartbeat must be newer than five minutes and match the current key ID/mode before hosted initiation or resume is offered. A stopped/unconfigured worker therefore disables new production payments. Callback verification and status recovery still work. Check worker response `pending`, event backlog age, heartbeat and `payment_failure` diagnostics; alert on any MANUAL_REVIEW, repeated deferred reconciliation or backlog older than five minutes. Batch throughput and scheduler frequency must be reviewed for store volume.

No scheduler or production payment credentials are installed by this code change. Vercel deployment alone does not establish an operational payment system.

Dashboard configuration

1. Select the correct Razorpay mode and enable automatic capture according to the client's settlement policy. This code never treats authorization as capture and does not call an automatic capture endpoint.
2. Configure the webhook URL: `https://www.parinayasarees.com/api/payments/webhook`.
3. Set its independent mode-specific secret and subscribe to `payment.authorized`, `payment.captured`, `payment.failed`, `order.paid`, `refund.created`, `refund.processed`, `refund.failed`.
4. Enable the account's approved UPI, cards and net-banking methods. The Standard Checkout window displays the methods actually available to that account/device; this UI does not assert that account activation is complete.
5. Check webhook delivery acknowledgments, durable acceptance and subsequent processing in TEST before LIVE. The public endpoint must bypass deployment protection that would block Razorpay; it authenticates each original payload with HMAC. Do not disable signature checking to solve delivery problems.

Official references: Standard Checkout integration, webhook validation, Orders API, order payments, Vercel function Web handlers.

API and authorization

One `api/payments/[action].ts` Node function provides these routes. Vercel Web GET/POST exports preserve the request stream and original webhook bytes; the Next.js `bodyParser` configuration is not used or assumed to work in Vite. Vercel's redundant `action` route query is removed only when it exactly matches the path; other query inputs remain strictly rejected.

Route	Purpose
GET `capabilities`	Authenticated, account-specific safe availability
POST `create-order`	Trusted review, one reservation and one provider creation attempt
POST `verify`	Timing-safe callback HMAC followed by authoritative provider reconciliation
GET `status?id=…`	Owned order recovery; without ID, recover the account's active payment
GET `status?operationId=…`	Recover a lost create response using token UID and the original operation
POST `resume`	Reconcile, then return the same existing provider order only when safe
POST `webhook`	Verify original-body HMAC; atomically accept a minimized event
POST `reconcile`	Both admin claim and enabled access record; audit and provider reconciliation
POST `worker`	Scheduler secret, no browser access

Customer endpoints use revocation-aware Firebase ID-token verification. UID comes from the verified token, never a posted UID. Admin reconciliation additionally requires `admin:true` and `adminAccess/{uid}.enabled:true`, including an access record check inside its audit transaction. Inputs are strictly bounded. Request bodies are limited to 64 KiB; authenticated calls are limited to 30 per minute per UID, shared across instances via Firestore. There is no cookie-authenticated mutation; Origin and Fetch Metadata reject cross-site browser requests. Production requires HTTPS and allows only the two approved storefront origins.

Responses are private/no-store. Logs contain fixed reason codes and internal references, never tokens, signatures, secrets, full provider payloads, names, addresses, phone numbers, card details, OTPs or CVV. Provider error descriptions are not logged. Never enable HTTP debug logging that serializes authorization.

Data, state and idempotency

Existing `orders/{id}` owns the immutable INR/paise purchase snapshot and safe payment status. `paymentAttempts/{id}` stores mode/key-ID binding, provider order mapping, creation outcome, accepted Bag quantities, due time and minimized refund evidence. Additional private server-only collections are `paymentProviderOrders`, `paymentCheckoutLocks`, `paymentEvents`, `paymentRateLimits`, `paymentReconciliation`, and `paymentOperations`. No credentials are stored there. The existing default-deny Firestore rules already deny browser reads/writes to these collections, including ordinary administrator SDK clients. Orders remain server-written. No rule publication is required. The mode-aware worker requires the two collection-scope composite indexes listed in `PAYMENT_LIVE_CUTOVER.md`: `mode ASC, keyId ASC, nextCheckAt ASC` on `paymentEvents` and `paymentAttempts`. Both indexes were created and verified READY in Production on 9 November 2026; their IDs and matching read-only query results are in `PAYMENT_LIVE_CUTOVER.md`. For another environment, create only missing exact definitions and wait for READY before deploying that worker change. Filtering before the batch limit prevents TEST or rotated-key backlog from consuming the active credential's processing capacity.

Payment status is separate from fulfillment. States include CREATED, PENDING, AUTHORIZED, CAPTURED, FAILED, CANCELLED, VERIFICATION_PENDING, REFUND_PENDING, PARTIALLY_REFUNDED, REFUNDED and MANUAL_REVIEW. The customer's sanitized projection omits provider IDs and diagnostics. Administrators see actual mappings, timestamps, method, amount, refunds, reservation state and safe review reason.

The existing transaction validates current products/variants, category visibility, shipping, coupon/sequence eligibility and points. It reserves inventory, coupon usage and applied points atomically. The reservation is 30 minutes. Applied points remain reserved; earnings still occur only on eligible Delivered transitions. The frozen snapshot is honored after payment begins; later price, shipping or coupon-setting changes cannot rewrite the amount already submitted to Razorpay. Before initiation, changed reviews require fresh customer approval.

One account lock blocks a competing checkout, including COD, while its online payment is unresolved. Duplicate clicks and the original operation reuse the same internal/provider order. Provider creation is claimed before the one POST; a lost response is reconciled by its unique receipt and internal note through a bounded, fully paginated Orders API window. An unknown outcome is never blindly POSTed again. Too many results, missing evidence or ambiguity goes to manual review.

Only a matching captured payment, a paid matching provider order and exact INR amount can confirm fulfillment. Capture, history, notification and selective Bag cleanup commit atomically. Changed Bag quantities remain saved. Repeated, concurrent or older authorized/failed events cannot downgrade capture or deduct stock again. Webhooks are acknowledged only after durable event-ID/hash acceptance; slower provider work runs in the recoverable worker. A failed internal transaction retains the reservation for reconciliation instead of claiming success.

Expired reservations are released once only after provider checks show no capture or authorization. Browser dismissal/failure never releases stock. Unknown create outcomes and authorized payments stay held. A later capture after release is MANUAL_REVIEW and cannot fulfill; no automatic refund or stock availability is promised. The owner must review provider evidence and inventory before deciding whether to refund or arrange fulfillment. The reconciliation button never forces an uncertain record to paid or bypasses an inventory hold.

Refund readiness and limits

This release deliberately has no refund-initiation button. Existing manual COD collection/refund commands reject online orders. A provider-confirmed processed refund changes the online refund total; pending or failed refund events never claim money was returned. Processed refund evidence cannot be downgraded by an older notification. Full/partial cash refunds are reported separately from delivery.

A refund with loyalty requires merchandise/shipping allocation under the existing points policy. Until a trusted allocation workflow is approved, it places a reconciliation hold on that order and points spending for the customer, protecting against spending unreversed earnings. There is no UI switch to clear that hold. Refund initiation, automatic inventory returns and automated loyalty allocation are **not** claimed complete. Future refund initiation must add dual-admin authorization, supported provider refund idempotency, refundable-amount checks and allocation with an audited, idempotent trusted repair. Do not edit balance numbers or clear review flags to bypass the required accounting.

Recovery and customer communication

Customers see pending, authorization, verified capture and actual refund states in checkout, order history and order details. Pending orders never show a success celebration. Closing Checkout or losing connectivity leads to Check payment status, not a new charge. Continue existing payment reuses only a reconciled unexpired provider order with no authorization/capture. No order confirmation is inferred from a callback signature, browser route or HTTP 200 alone. Original operation IDs are persisted before requests and scoped to the account. Another device can recover the active attempt from the server. Cleared storage does not clear the server lock.

Only existing on-screen and admin notifications are implemented. There is no claim of automated transactional email, SMS or WhatsApp sending from this integration.

Verification and release procedure

Run `npm test`, `npm run lint`, `npm run build`, `npm run test:catalog-runtime`, `npm run test:payments-runtime` and the loopback `payments-emulator` plus existing coupon, loyalty, catalog, admin/customer/category rules and orders regression suites through `npm run test:client-changes`. `scripts/payments-browser.test.py` intercepts the gateway and covers the approved checkout/result layout at 320, 390, 768 and 1440 pixels. It never contacts Razorpay. The packaged runtime can also be exercised inside the loopback suite with `PAYMENT_RUNTIME_EMULATOR=1`; it verifies original-byte HMAC and durable acceptance using compiled JS, without tsx at runtime.

For actual Razorpay TEST acceptance, use approved synthetic test accounts/products in an isolated test Firebase dataset/environment, complete Standard Checkout's success and decline scenarios, test callback loss/dismissal, use the Dashboard's webhook testing/retry controls, inspect provider order/payment/refund evidence and confirm one stock/points/coupon transition. Never create production business records just to validate payment listing or a deployment.

Before LIVE: client account activation and required methods enabled; live credentials installed securely; matching live webhook configured; automatic capture confirmed; scheduler healthy in the selected mode; TEST matrix accepted; alerts and a named reconciliation owner assigned; exposed historical credentials rotated; then set the explicit live activation flag, redeploy and verify a client-authorized real transaction separately. A READY deployment does not prove settlement or webhook processing. Refund/accounting automation limits above must be accepted before launch.

Rotation and previously committed credentials

The pre-existing tracked `.env.example` contained a Firebase service-account private key and Cloudinary secret. This change replaces the template with placeholders, but removing current contents cannot erase old Git history. Treat those credentials as exposed: rotate/revoke the Firebase service-account key and rotate the Cloudinary API secret, update secure deployment/local settings, redeploy, and verify their consumers. Do not publish old diffs, copy keys to tickets, or rewrite shared Git history without a separately coordinated plan.

For Razorpay rotation, disable new initiation, drain/reconcile in-flight attempts, retain controlled access to the original account/key/secret for unresolved attempts, rotate webhook configuration in a coordinated maintenance window and test delivery before re-enabling. Attempt key-ID binding rejects accidental reconciliation with different credentials. This implementation does not silently accept old signatures or switch modes for old attempts. Investigate old unprocessed events before removing their key/secret. Never automatically charge an old pending attempt again.

Implementation verification record (6 November 2026)

- 185 unit tests; TypeScript (`npm run lint`); production Vite build passed. The existing large-chunk build warning remains.
- Emulator regressions passed for payments, admin/customer/category rules, catalog, login appearance, shopping, addresses/shipping/settings, orders, coupons/sequence, loyalty and Delivered awards, inventory, business reports, campaigns/banners, content/contact/enquiries/About, customer directory, order details and delivery.
- Real loopback HTTP-handler tests with mocked Razorpay cover one provider POST, strict ownership/dual admin, signatures versus capture, duplicate/concurrent callbacks and events, declines, unknown create recovery, restart, last-variant races, expired/late capture, coupon/shipping/stock changes, internal-commit failure, full/partial refunds, direct-write denial, rate limits, mode rotation and hosted worker gating.
- Browser checks passed for payment/checkout, Bag/Wishlist, customer order states, Admin order list/details and customer directory, theme/offers and campaign navigation. Payment views were checked at 320, 390, 768 and 1440 pixels. Blocked scripts, dismissal, duplicate clicks, missing-create recovery, refresh and lost verification never displayed a false confirmation or created a new charge.
- The pinned Vercel Node builder produced executable Node 24 function artifacts; local Node 22 invoked the compiled JS without tsx. The original-byte webhook signature/durable-acceptance test passed through the compiled Web exports.
- Frontend output has no private-key markers or matches for the examined historical secret values. The current env template has placeholders only. Historical Git exposure still requires the credential rotation described above.

No real Razorpay TEST transaction was executed: TEST credentials are unavailable. No gateway credentials, worker secret, TEST UID allowlist or live activation flag were present in the inspected Vercel Production variable-name inventory. The external scheduler has not been provisioned. Production Firestore rule equality could not be inspected with available local Admin credentials; verify that the deployed rules deny all client writes to payment/order records before enabling payments. The local least-privilege rules pass emulator tests and need no new definitions for this feature. No rules, indexes or business records were published or modified by implementation checks.